STT Engine Comparison - Mobile App Integration

WhisperLiveKit vs whisper.cpp vs openai/whisper

Dimension	WhisperLiveKit	whisper.cpp	openai/whisper
Mobile runtime	None — Python server only	**Official iOS/Android bindings, runs on-device via C API/Swift/Kotlin**	None — pure PyTorch, no mobile export path
Offline capable	No, needs server	**Yes**	No
Binary / footprint on phone	N/A (client is thin)	**Small, quantized ggml models (tiny/base run fine on phone CPU/NPU)**	N/A — PyTorch runtime too heavy for mobile even if forced on-device
Server you have to run/ops	Yes — FastAPI + torch/faster-whisper, GPU/Metal helps	**None needed (can skip server entirely)**	Yes — same ops burden as WhisperLiveKit, with none of its session features
Live multi-viewer / share-link feature	**Already built (session tokens, pause/latency tuning done)**	Not built — you'd build a server in front of it anyway to get this	Not built — same gap, and no streaming primitives to build on
Mobile client complexity	Thin: open WebSocket, stream PCM, render JSON	Thin if on-device (native lib call); thin-over-network if a server is added instead	Thin-over-network only

Verdict

If the app needs the share/viewer feature (mic, share button, viewer side watching a live session), that requires a server-backed session regardless of engine — **WhisperLiveKit** is the easiest fit, since the viewer/share-link mechanism and latency tuning are already built and tested.

If offline capability or avoiding server hosting/ops matters more than the share/viewer feature, **whisper.cpp** is the easiest to actually embed in the app — real mobile SDKs exist, runs with zero network dependency. The share-link/viewer feature disappears unless a server is added anyway.

openai/whisper is the weakest choice either way: no mobile path, and server-side it has none of the streaming/session primitives.